

CAN

一、CAN介绍

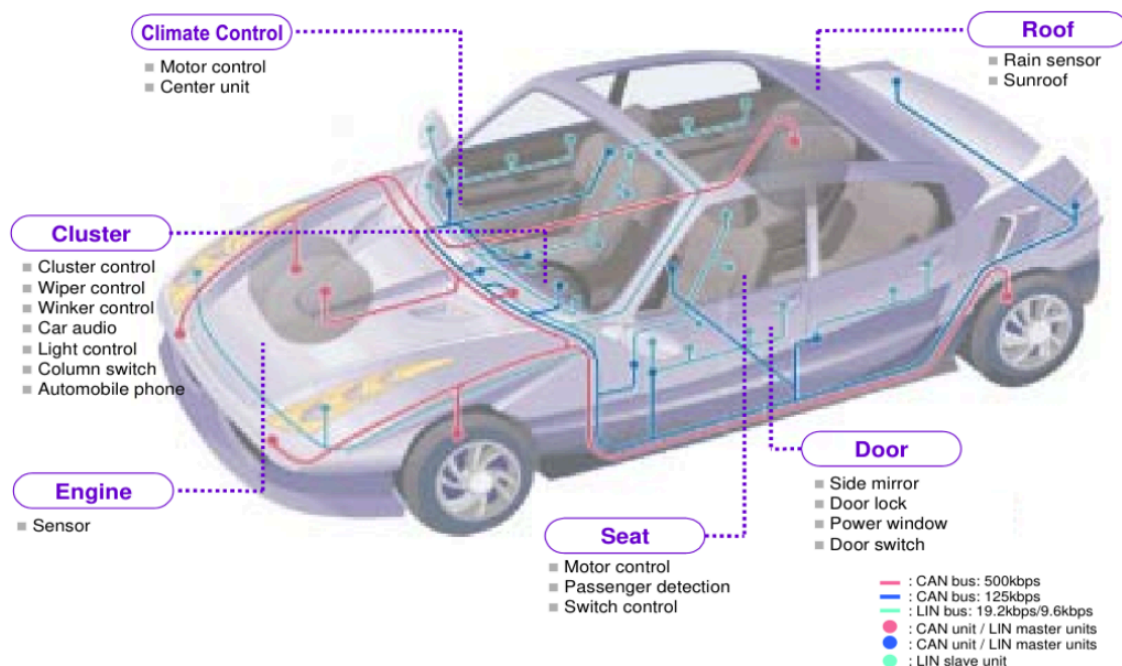
can是博世公司在1986年开发的串行通信协议，can的全称是Controller Area Network的缩写，翻译过来就是控制器域网络。can应用最多的就是汽车领域，为了满足汽车减少线束的数量应运而生。

局域网大家都不陌生，比如在同一个局域网的电脑可以相互通信。在汽车电子中，ECU是最小的控制模块，为了让不同的ECU模块可以相互通信，就开发了can总线。使用can总线实现ECU之间的局域网通信。

CAN 最初出现在80年代末的汽车工业中，由德国 Bosch 公司最先提出。

当时，由于消费者对于汽车功能的要求越来越多，而这些功能的实现大多是基于电子操作的，这就使得电子装置之间的通讯越来越复杂，同时意味着需要更多的连接信号线。提出 CAN 总线的最初动机就是为了解决现代汽车中庞大的电子控制装置之间的通讯，减少不断增加的信号线。于是，他们设计了一个单一的网络总线，所有的外围器件可以被挂接在该总线上。

1993年，CAN 已成为国际标准 ISO11898(高速应用)和 ISO11519（低速应用）。CAN 是一种多主方式的串行通讯总线，基本设计规范要求有高的位速率，高抗电磁干扰性，而且能够检测出产生的任何错误。当信号传输距离达到10Km 时，CAN 仍可提供高达50Kbit/s 的数据传输速率。由于 CAN 总线具有很高的实时性能，现在，CAN 的高性能和可靠性已被认同，并被广泛地应用于工业自动化、船舶、医疗设备、工业设备等方面



can发展历史：

1983年：Bosch开始研究汽车网络技术

1986年：Bosch正式发布can协议

1987年：英特尔和飞利浦先后推出can控制器芯片

1991年：Bosch发布can 2.0规范，同年can总线最先在奔驰S级轿车上实现

1993年：ISO发布can国际标准 ISO-11898

2015年: can fd ISO标准化 (在canfd没有标准化之前, 市面上已经有不少的canfd产品, 但都是属于非标准化的canfd产品)

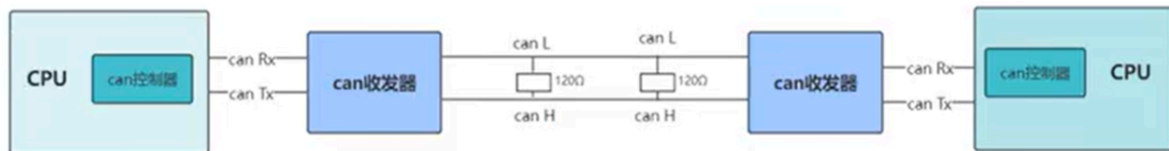
2020年: can xl问世

二、CAN硬件连接

ISO-11898 (高速, 通信速率为125kbit/s到1Mbit/s)

ISO-11519 (低速, 通信速率低于125kbit/s)

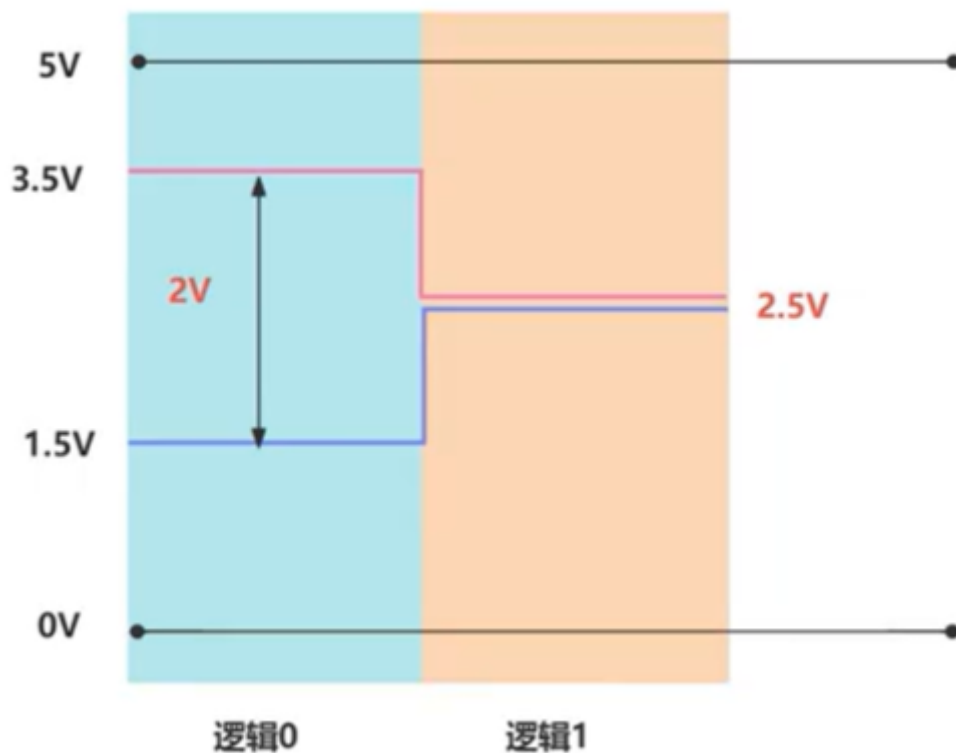
高速CAN的连接方式(CPU带CAN控制器):



can总线两端120Ω为终端电阻, 是为了消除总线上的信号反射 (两根信号线形成回路, 形成闭环结构的CAN总线网络)

注: 若CPU没有CAN控制器可以通过SPI转CAN芯片实现

三、电气属性



高速CAN (ISO-11898) :

当CAN_H和CAN_L电压相同 (CAN_H=CAN_L=2.5V) 时, 为逻辑"1"(隐性状态)

当CAN_H和CAN_L电压相差2V (CAN_H=3.5V,CAN_L=1.5V) 时, 为逻辑"0" (显性状态)

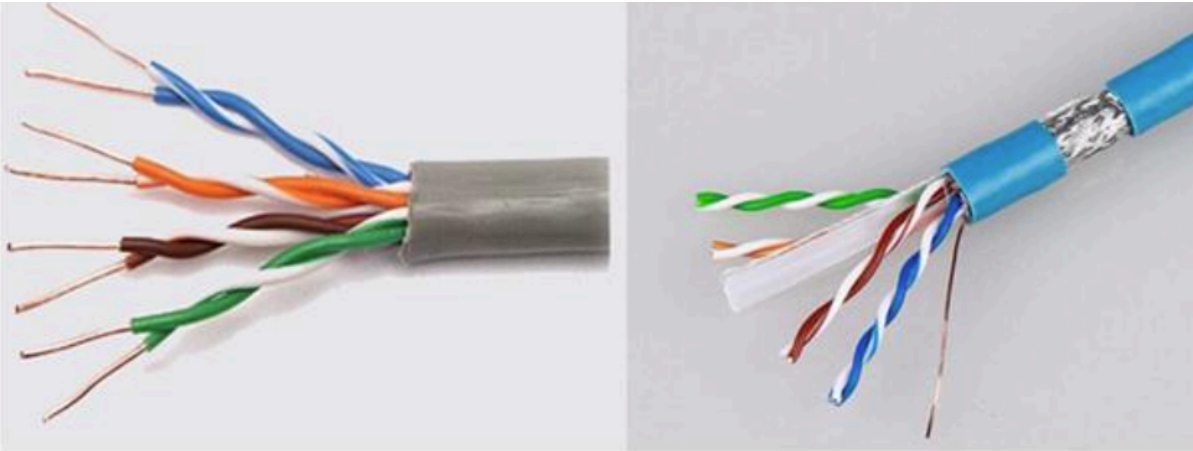
低速CAN (ISO-11519) :

当CAN_H和CAN_L电压差小于0V时，为逻辑"1"(隐性状态)

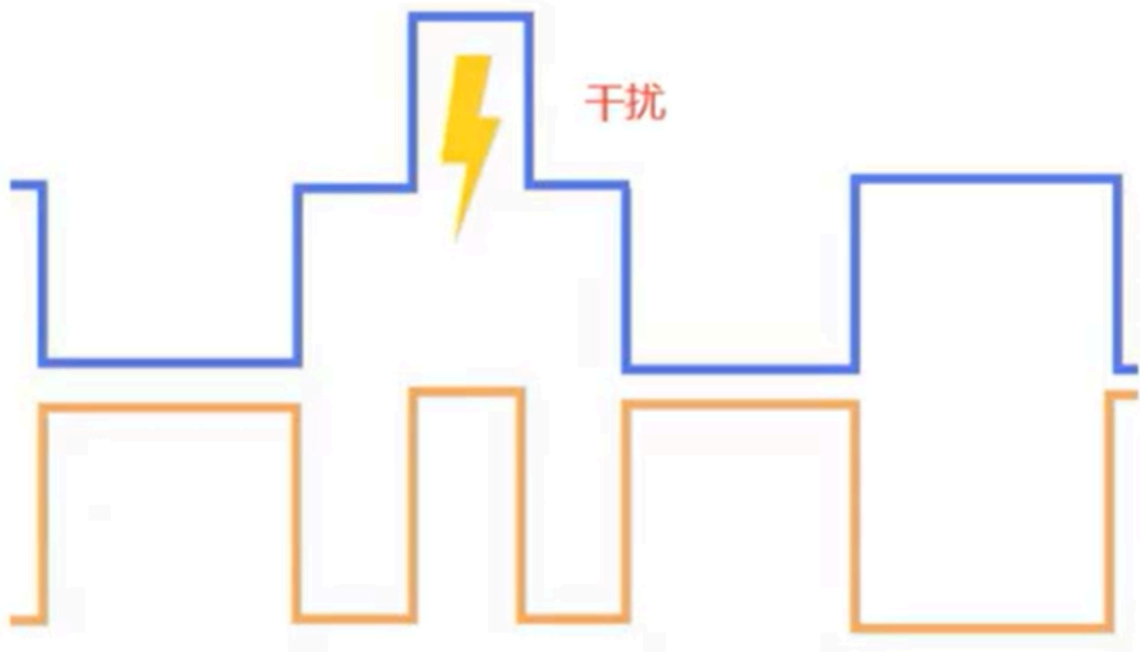
当CAN_H和CAN_L电压差大于2V时，为逻辑"0"（显性状态）

物理层	ISO 11898(High speed)						ISO 11519-2(Low speed)						
通信速度*1	最高 1Mbps						最高 125kbps						
总线最大长度*2	40m/1Mbps						1km/40kbps						
连接单元数	最大 30						最大 20						
总线拓扑*3	隐性			显性			隐性			显性			
	Min	Nom	Max.	Min.	Nom	Max.	Min	Nom.	Max.	Min.	Nom.	Max.	
	CAN_High (V)	2.00	2.50	3.00	2.75	3.50	4.50	1.60	1.75	1.90	3.85	4.00	5.00
	CAN_Low (V)	2.00	2.50	3.00	0.50	1.50	2.25	3.10	3.25	3.40	0.00	1.00	1.15
	电位差 (H-L)(V)	-0.5	0	0.05	1.5	2.0	3.0	-0.3	-1.5	-	0.3	3.00	-
双绞线（屏蔽/非屏蔽）						双绞线（屏蔽/非屏蔽）							
闭环总线						开环总线							
阻抗(Z): 120Ω (Min.85Ω Max.130Ω)						阻抗(Z): 120Ω (Min.85Ω Max.130Ω)							
总线电阻率(r): 70mΩ/m						总线电阻率(Γ): 90mΩ/m							
总线延迟时间: 5ns/m						总线延迟时间: 5ns/m							
终端电阻: 120Ω (Min.85Ω Max.130Ω)						终端电阻: 2.20kΩ (Min.2.09kΩ Max.2.31kΩ)							
						CAN_L 与 GND 间静电容量 30pF/m							
						CAN_H 与 GND 间静电容量 30pF/m							
						CAN_L 与 GND 间静电容量 30pF/m							

CAN总线的两根信号线通常采用的是双绞线，如下图所示，传输的是差分信号，通过两根信号线的电压差CANH-CANL来表示总线电平。以差分信号传输信息具有抗干扰能力强，能有效抑制外部电磁干扰等优点，这也是CAN总线在工业上应用广泛的一个原因。



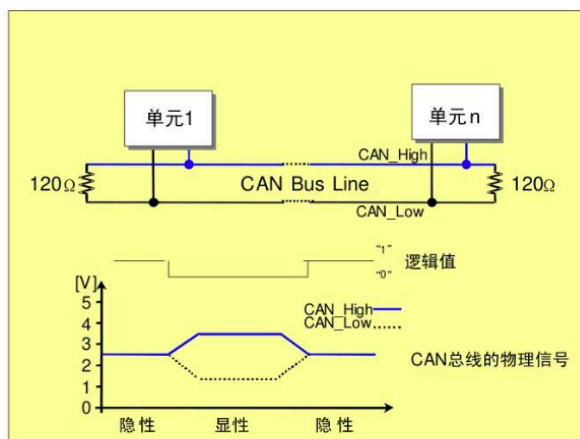
注：（左边为非屏蔽型双绞线，右边为屏蔽型双绞线）



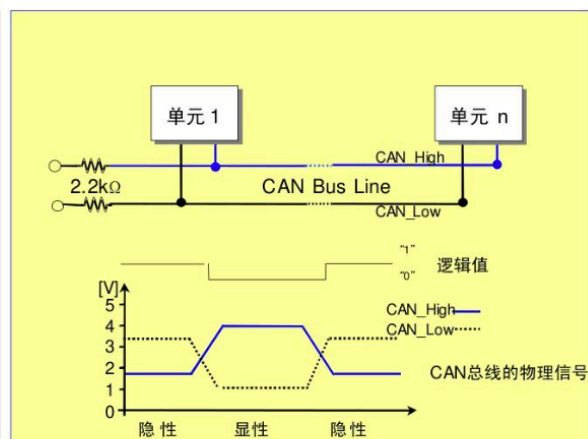
注：干扰信号对双绞线的干扰是等幅值，等相位，等频率的，所以双绞线抗干扰能力更好

所有can总线速率最快可达到1Mbit/s，此时传输距离最远可达到40m。

CAN总线上的一个终端设备称为一个节点（Node），在CAN网络中，没有主设备和从设备的区别。一个CAN节点的硬件部分一般由CAN控制器和CAN收发器两个部分组成。CAN控制器负责CAN总线的逻辑控制，实现CAN传输协议；CAN收发器主要负责MCU逻辑电平与CAN总线电平之间的转换



【ISO11898(125K~1Mbps)】



【ISO11519-2(10K~125kbps)】

四、CAN总线特点

- 实时性：** CAN总线具有优越的实时性能，适用于需要及时传输数据的应用，如汽车控制系统、工业自动化等。仲裁机制和帧优先级的设计保证了低延迟和可预测性。
- 多主机系统：** CAN支持多主机系统，多个节点可以同时发送和接收数据。这种分布式控制结构使得系统更加灵活，适用于复杂的嵌入式网络。CAN总线上的节点既可以发送数据又可以接收数据，没有主从之分。但是在同一个时刻，只能由一个节点发送数据，其他节点只能接收数据。

3. **差分信号传输**: CAN使用差分信号传输, 通过两个线路 (CAN_H和CAN_L) 之间的电压差来传递信息。这种差分传输方式提供了良好的抗干扰性能, 使得CAN总线适用于工业环境等有电磁干扰的场合。
4. **仲裁机制**: CAN总线采用非破坏性仲裁机制, 通过比较消息标识符的优先级来决定哪个节点有权继续发送数据。这种机制确保了总线上数据传输的有序性, 避免了冲突。
5. **广播通信**: CAN总线采用广播通信方式, 即发送的数据帧可以被总线上的所有节点接收。这种特性有助于信息的共享和同步, 同时减少了系统的复杂性。
6. **低成本**: CAN总线的硬件成本相对较低, 适用于大规模的系统集成。由于CAN控制器在硬件上实现了仲裁机制, 无需额外的主机处理器, 减小了成本和复杂性。
7. **灵活性**: CAN协议灵活适应不同的应用场景, 支持不同的波特率和通信速率。这使得CAN总线可以被广泛用于各种嵌入式系统, 从低速的传感器网络到高速的汽车控制系统。
8. **错误检测和处理**: CAN总线具有强大的错误检测和处理机制。通过CRC检查和其他错误检测手段, CAN能够识别和处理传输过程中可能发生的错误, 提高了通信的可靠性。
9. **多种帧类型**: CAN总线上的节点没有地址的概念。CAN总线上的数据是以帧为单位传输的, 帧又分为数据帧、遥控帧等多种帧类型, 帧包含需要传输的数据或控制信息。
10. **线与逻辑**: CAN总线具有“线与”的特性, 也就是当由两个节点同时向总线发送信号时, 一个是发送显性电平 (逻辑0), 另一个发送隐性电平 (逻辑1), 则总线呈现为显性电平。这个特性被用于总线仲裁, 也就是哪个节点优先占用总线进行发送操作。
11. **特定标识符**: 每一个帧有一个标识符 (Identifier, 以下简称ID)。ID不是地址, 它表示传输数据的类型, 也可以用于总线仲裁时确定优先级。例如, 在汽车的CAN总线上, 假设用于碰撞检测的节点输出数据帧ID为01, 车内温度检测节点发送数据帧的ID为05等。
12. **滤波特性**: 每个CAN节点都接收数据, 但是可以对接收的帧根据ID进行过滤。只有节点需要的数据才会被接收并进一步处理, 不需要的数据会被自动舍弃。例如, 假设安全气囊控制器只接受碰撞检测节点发出的ID为01的帧, 这种ID的过滤时有硬件完成的, 以便安全气囊控制器在发送碰撞时能及时响应。
13. **半双工**: CAN总线通信时半双工的, 即总线不能同时发送和接收。在多个节点竞争总线进行发送时, 通过ID的优先级进行仲裁, 竞争胜出的节点继续发送, 竞争失败的节点立刻转入接收状态。
14. **无时钟信号**: CAN总线没有用于同步的时钟信号, 所以需要规定CAN总线通信的波特率, 所以节点都是用同样的波特率进行通信。

五、CAN位时序和波特率

一个CAN网络需要规定一个通信的波特率, 各节点都以相同的波特率进行数据通信。位时序指的是一个节点采集CAN总线上的一个位数据的时序。通过位时序的控制, CAN总线可以进行位同步, 以吸收节点时钟差异产生的波特率误差, 保证接收数据的准确性。



标称位时间 (Nominal Bit Time, NBT) 指的是传输一个位数据的时间, 用于确定CAN总线的波特率。这个事件被分成了3段。

1. 同步段 (SYNC_SEG)：在这个时间段内，总线上应该发送一次位信号的跳变。如果节点在同步段检测到总线上的一个跳变沿，就表示节点与总线是同步的。同步段长度固定为1个tq。
tq (time quantum) 被称为时间片，tq由CAN控制器的时钟频率fcan决定。在STM32F407中，两个CAN控制器在APB1总线上，CAN控制器有预分频器，APB1总线的时钟信号PCLK1经分频后得到fcan。
 2. 位段1 (Bit Segment 1, BS1)：定义了采样点的位置。在BS1结束的时间点对总线采样，得到的电平就是这个位的电平。BS1的初始长度是1到16个tq，但它的长度可以在再同步 (resynchronization) 的时候被自动加长，以补偿各节点频率差异导致的正相位漂移。
 3. 位段2 (Bit Segment 2, BS2)：定义了发送点的位置。BS2的初始长度是1到8个tq，再同步时可以被自动缩短，以补偿负相位漂移。
- CAN控制器可以自动对位时序进行再同步，再同步时自动调整BS1和BS2的长度，位段加长或缩短的上线称为再同步跳转宽度 (resynchronization Jump Width, SJW)，SJW的取值是1到4个tq。CAN总线的波特率就由标称位时间长度NBT决定，而NBT时位时序3个段的时间长度和，即
- \$\$
- $$NBT = (1 + m + n) \times t_q$$
- \$\$

$$Baudrate = \frac{1}{NBT}$$

六、CAN总线协议

CAN 协议涵盖了 ISO 规定的 OSI 基本参照模型中的传输层、数据链路层及物理层。

ISO/OSI 基本参照模型		各层定义的主要项目
软件控制	7 层：应用层	由实际应用程序提供可利用的服务。
	6 层：表示层	进行数据表现形式的转换。 如：文字设定、数据压缩、加密等的控制
	5 层：会话层	为建立会话式的通信，控制数据正确地接收和发送。
	4 层：传输层	控制数据传输的顺序、传送错误的恢复等，保证通信的品质。 如：错误修正、再传输控制。
	3 层：网络层	进行数据传送的路由选择或中继。 如：单元间的数据交换、地址管理。
硬件控制	2 层：数据链路层	将物理层收到的信号（位序列）组成有意义的数 据，提供传输错误控制等数据传输控制流程。 如：访问的方法、数据的形式。 通信方式、连接控制方式、同步方式、检错方式。 应答方式、通信方式、包（帧）的构成。 位的调制方式（包括位时序条件）。
	1 层：物理层	规定了通信时使用的电缆、连接器等的媒体、电气信号规格等，以实现设备间的信号传送。 如：信号电平、收发器、电缆、连接器等的形态。

【注】*1 OSI: Open Systems Interconnection（开放式系统间互联）

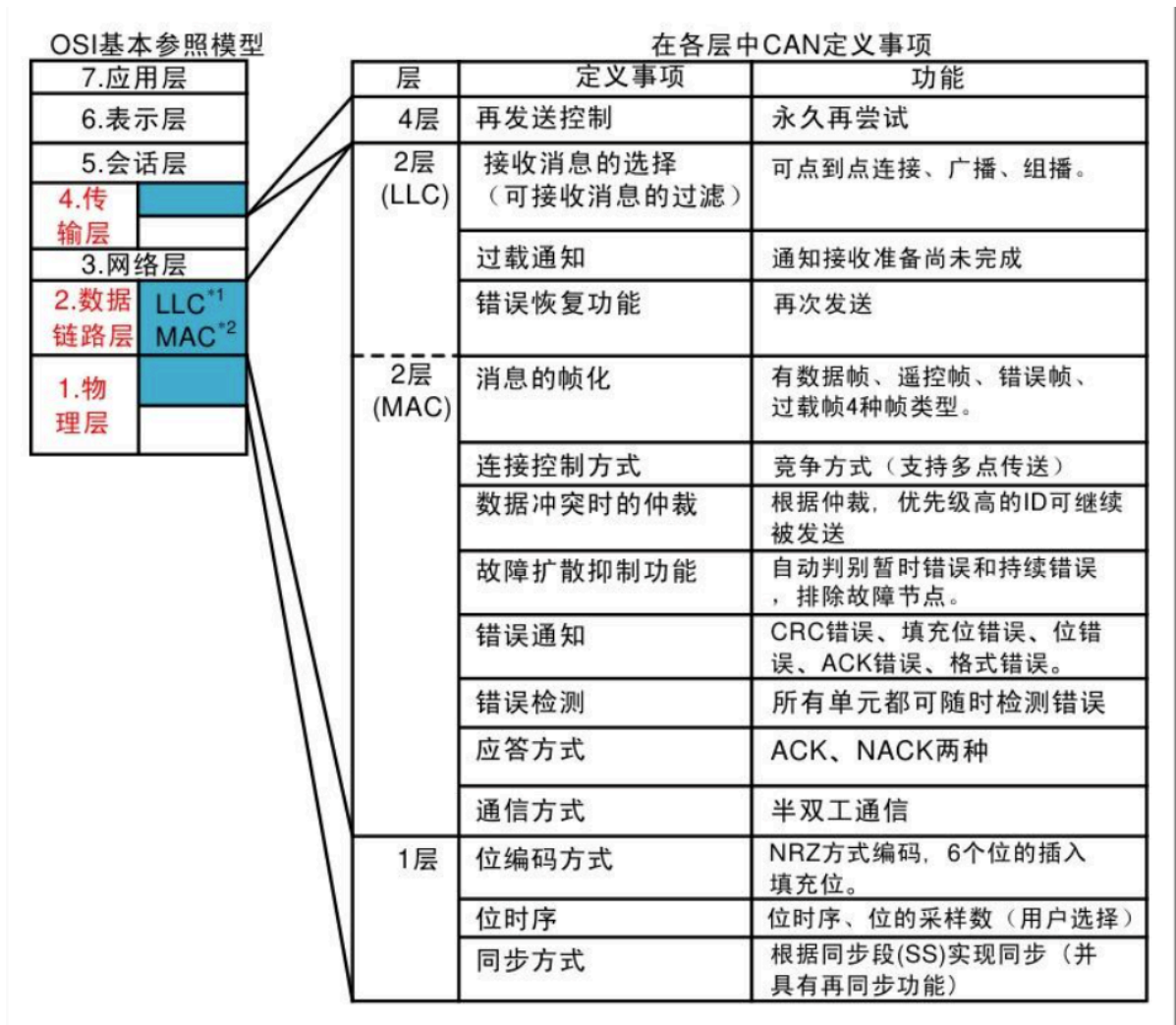


图8 ISO/OSI 基本参照模型和 CAN 协议

数据链路层分为 [MAC子层](#)和 [LLC子层](#), MAC子层是CAN协议的核心部分。数据链路层的功能是将物理层收到的信号组织成有意义的消息, 并提供传送错误控制等传输控制的流程。具体地说, 就是消息的帧化、仲裁、应答、错误的检测或报告。数据链路层的功能通常在CAN控制器的硬件中执行。在物理层定义了信号实际的发送方式、位时序、位的编码方式及同步的步骤。但具体地说, 信号电平、通信速度、采样点、驱动器和总线的电气特性、连接器的形态等均未定义。这些必须由用户根据系统需求自行确定。

七、CAN帧类型

CAN网络通信是通过5中类型的帧(Frame)进行的:

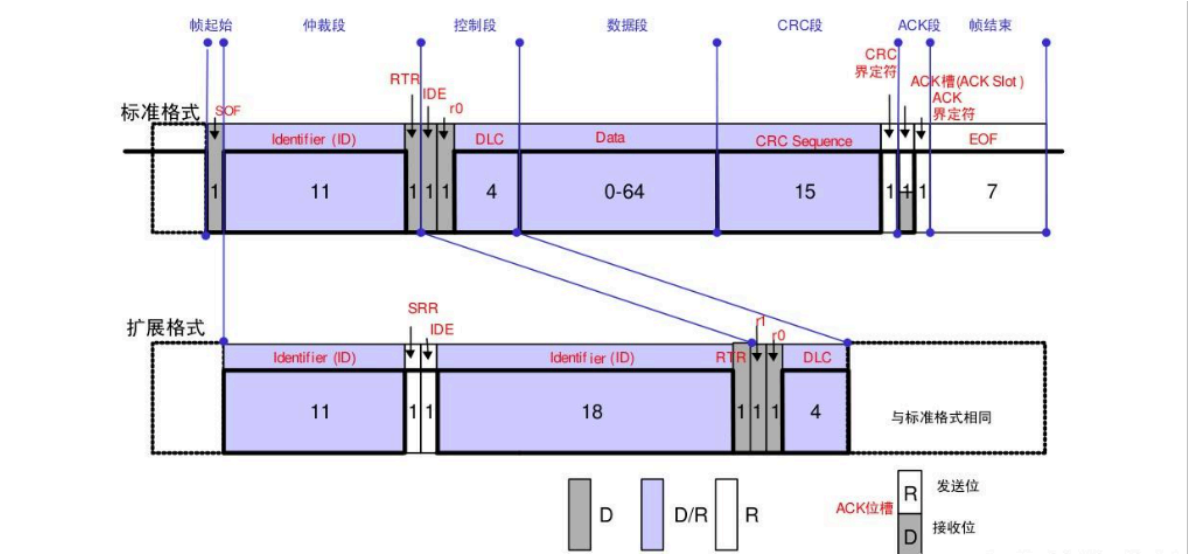
- 数据帧(Data frame)
- 遥控帧(Remote frame)
- 错误帧(Error frame)
- 过载帧(Overload frame)
- 帧间空间(Inter-frame space)

数据帧和遥控帧有标准格式和扩展格式两种格式。标准格式有11个位的标识符(Identifier: 以下称ID), 扩展格式有29个位的ID:

帧类型	帧用途
数据帧 (Data frame)	节点发送的包含 ID 和数据的帧，用于发送单元向接收单元传送数据的帧。
遥控帧 (Remote frame)	节点向网络上的其他节点发出的某个 ID 的数据请求，发送节点收到遥控帧后就可以发送相应 ID 的数据帧，
错误帧 (Error frame)	节点检测出错误时，向其他节点发送的通知错误的帧
过载帧 (Overload frame)	接收单元未做好接收数据的准备时发送的帧，发送节点收到过载帧后可以暂缓发送数据帧
帧间空间 (Inter-frame space)	用于将数据帧、遥控帧与前后的帧分隔开的帧

1. 数据帧(Data frame):

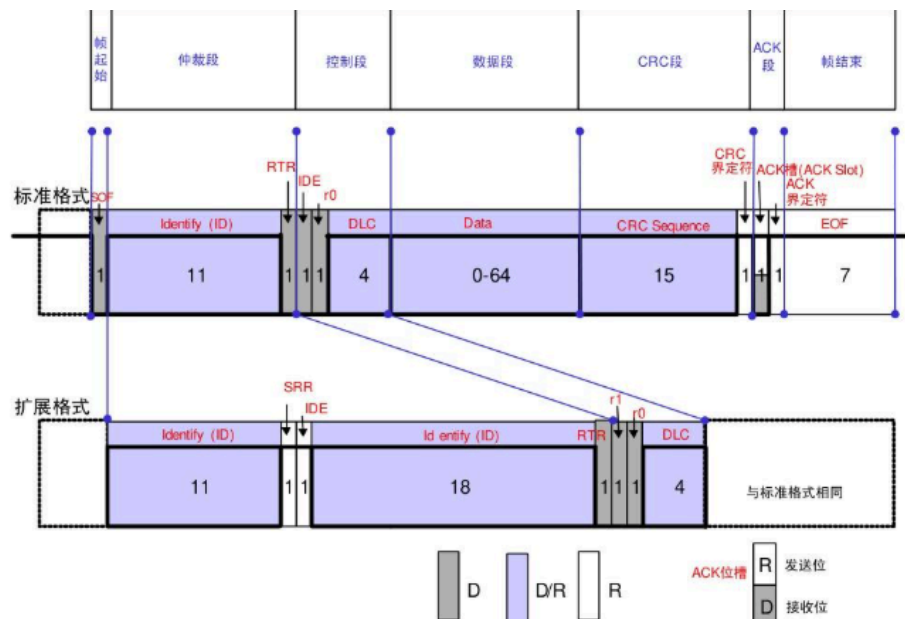
用于发送单元向接收单元发送数据的帧



数据帧由 7 个段构成。数据帧的构成如上图所示。

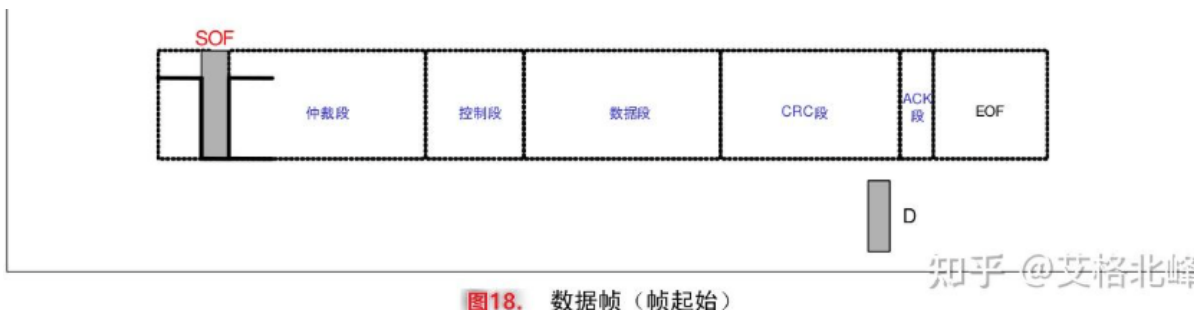
1. **帧起始:** 表示数据帧开始的段。
2. **仲裁段:** 表示该帧优先级的段。
3. **控制段:** 表示数据的字节数及保留位的段。
4. **数据段:** 数据的内容，可发送 0~8 个字节的数据。
5. **CRC 段:** 检查帧的传输错误的段。
6. **ACK 段:** 表示确认正常接收的段。
7. **帧结束:** 表示数据帧结束的段。

下面对帧的构成进行说明：



(1)帧起始（标准、扩展格式相同）

表示帧开始的段。1个位的显性位。



显性电平和隐性电平：

总线上的电平有显性电平和隐性电平两种。

总线上执行逻辑上的线"与"特性时，显性电平的逻辑值为'0'，隐性电平的逻辑值为'1'

"显性"具有"优先"的意味，总线上只要有一个单元输出显性电平，总线上即为显性电平。

"隐性"具有"包容"的意味，只有总线上的所有单元输出隐性电平，总线上才呈现隐性电平。

（显性电平比隐性电平更强）

(2)仲裁段

表示数据的优先级的段。标准格式和扩展格式在此的构成有所不同

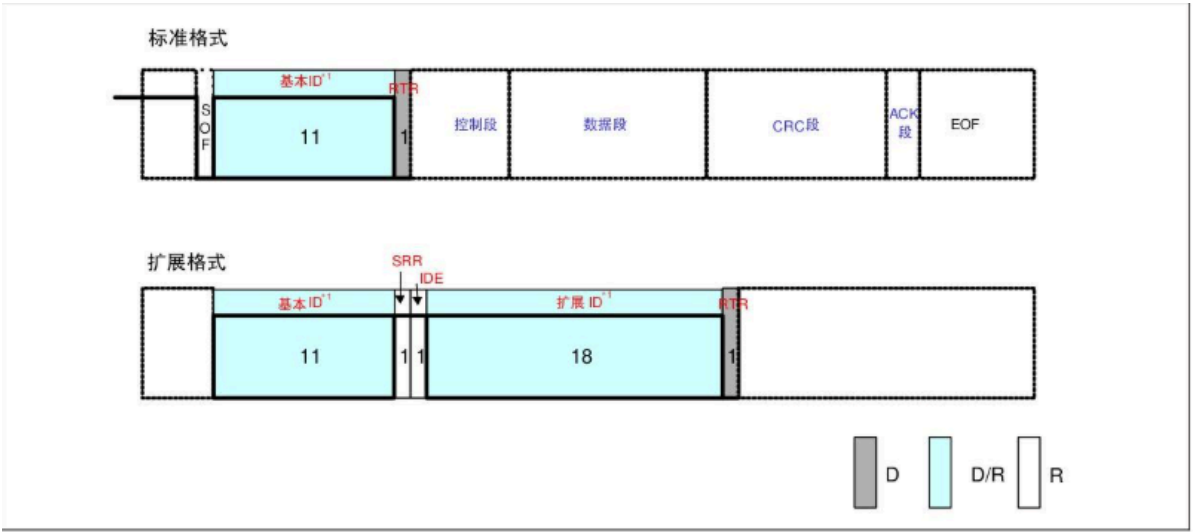


图19. 数据帧（仲裁段）

【注】 *1 ID
标准格式的ID有11个位。从ID28到ID18被依次发送。禁止高7位都为隐性。
（禁止设定：ID=1111111XXXX）
扩展格式的ID有29个位。基本ID从ID28到ID18，扩展ID由ID17到ID0表示。基本ID和标准格式的ID相同。禁止高7位都为隐性。（禁止设定：基本ID=1111111XXXX）

(3)控制段

控制段由6个位构成，表示数据段的字节数。标准格式和扩展格式的构成有所不同。

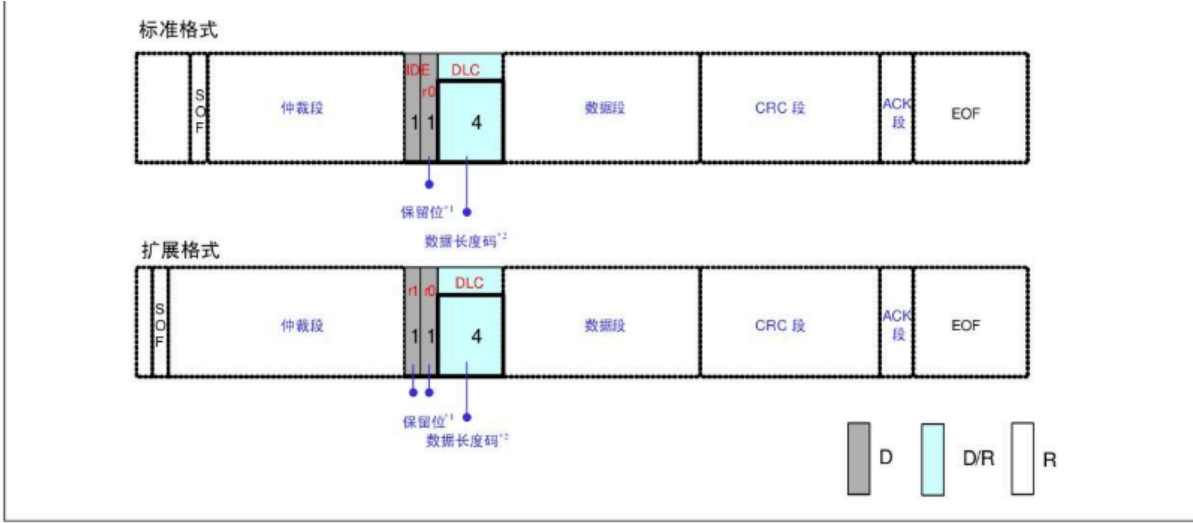


图20. 数据帧（控制段）

【注】 *1 保留位（r0、r1）
保留位必须全部以显性电平发送。但接收方可以接收显性、隐性及其任意组合的电平。
*2 数据长度码（DLC）
数据长度码与数据的字节数的对应关系如表8所示。
数据的字节数必须为0~8字节。但接收方对DLC=9~15的情况并不视为错误。

表5. 数据长度码和字节数的关系

数据字节数	数据长度码			
	DLC3	DLC2	DLC1	DLC0
0	D	D	D	D
1	D	D	D	R
2	D	D	R	D
3	D	D	R	R
4	D	R	D	D
5	D	R	D	R
6	D	R	R	D
7	D	R	R	R
8	R	D	D	D

“D”：显性电平 “R”：隐性电平

(4)数据段（标准、扩展格式相同）

数据段可包含 0 ~ 8 个字节的数据。从 MSB（最高位）开始输出

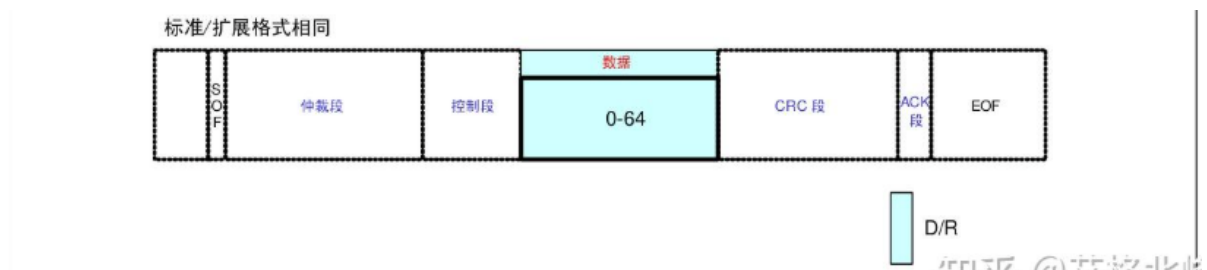


图21. 数据帧（数据段）

(5)CRC段（标准/扩展格式相同）

CRC 段是检查帧传输错误的帧。由 15 个位的 CRC 顺序和 1 个位的 CRC 界定符（用于分隔的位）构成。

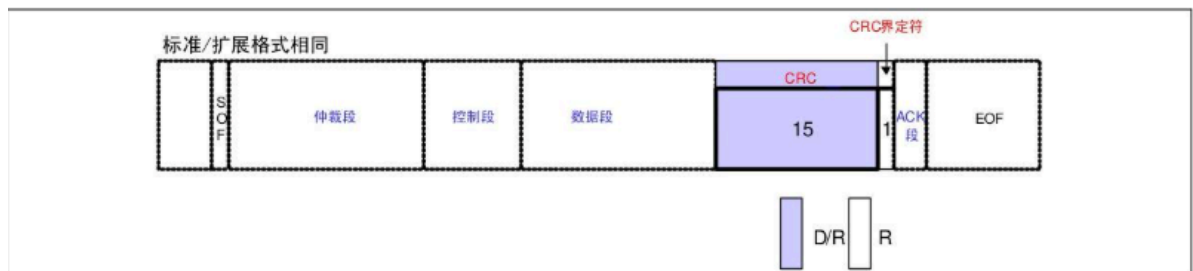


图22. 数据帧（CRC 段）

【注】 *1 CRC 顺序

CRC 顺序是根据多项式生成的 CRC 值，CRC 的计算范围包括帧起始、仲裁段、控制段、数据段。

接收方以同样的算法计算 CRC 值并进行比较，不一致时会通报错误。

(6)ACK段

ACK 段 (Acknowledge Bit) 用来确认是否正常接收。由 ACK 槽(ACK Slot)和 ACK 界定符 2 个位构成。

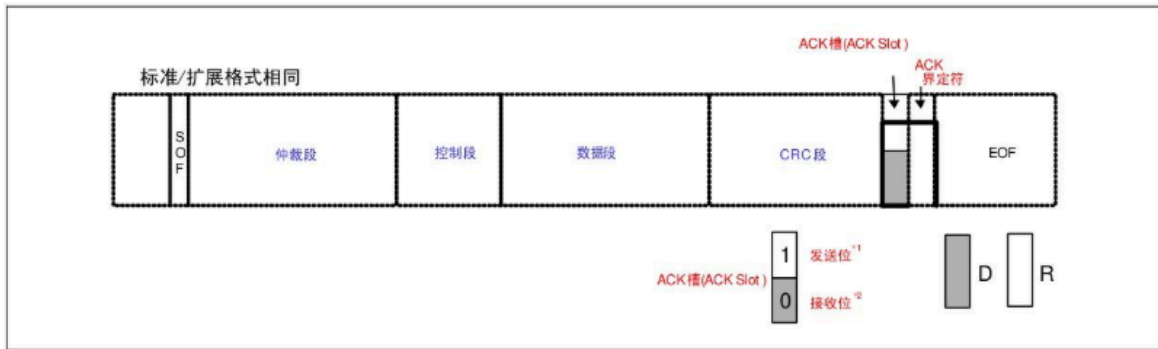


图23. 数据帧（ACK 段）

- 【注】
- *1 发送单元的 ACK 段
发送单元在 ACK 段发送 2 个位的隐性位。
 - *2 接收单元的 ACK 段
接收到正确消息的单元在 ACK 槽(ACK Slot)发送显性位, 通知发送单元正常接收结束。这称作“发送 ACK”或者“返回 ACK”。

发送 ACK

发送 ACK 的是在既不处于总线关闭态也不处于休眠态的所有接收单元中, 接收到正常消息的单元 (发送单元不发送 ACK)。所谓正常消息是指不含填充错误、格式错误、CRC 错误等。

(7)帧结束 (End of Frame, EOF)

帧结束是表示该帧的结束的段。由 7 个位的隐性位构成。

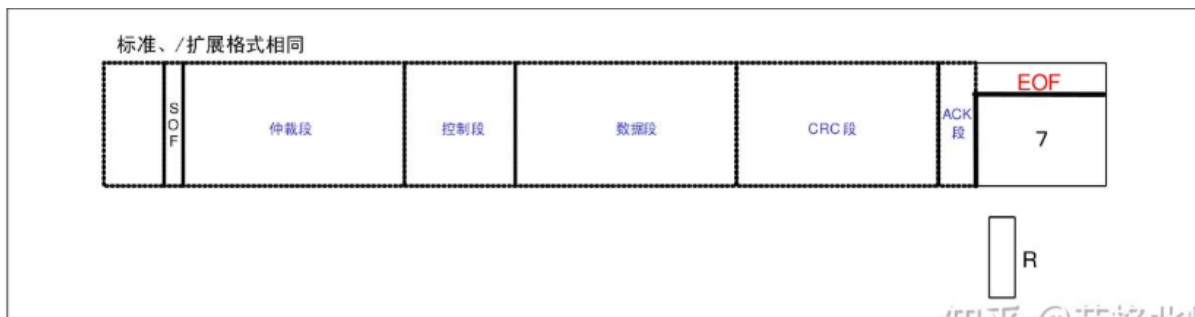
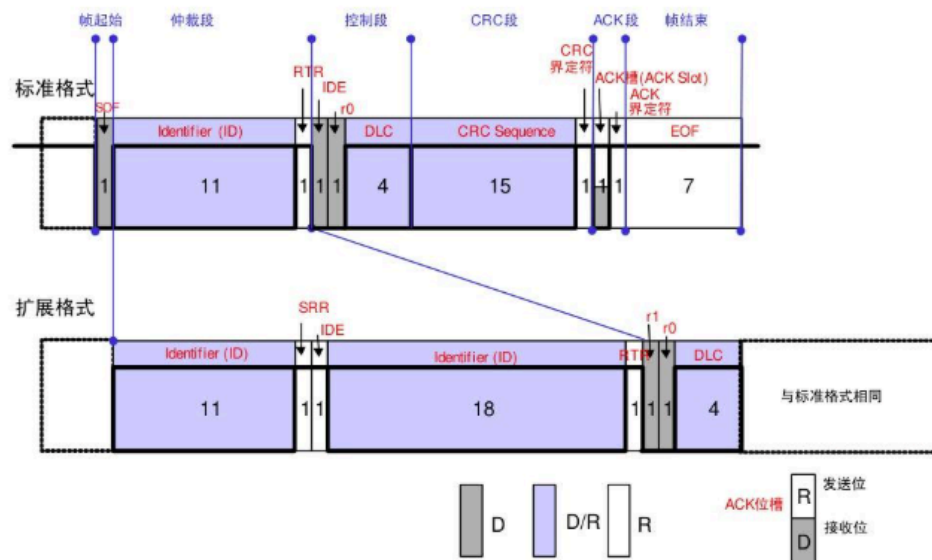


图24. 数据帧（帧结束）

2. 遥控帧(Remote frame):

用于接收单元向具有相同 ID 的发送单元请求数据的帧



接收单元向发送单元请求发送数据所用的帧。遥控帧由 6 个段组成。遥控帧没有数据帧的数据段。遥控帧的构成如图 所示。

1. **帧起始**：表示帧开始的段。
2. **仲裁段**：表示该帧优先级的段。
3. **控制段**：表示数据的字节数及保留位的段。
4. **CRC 段**：检查帧的传输错误的段。
5. **ACK 段**：表示确认正常接收的段。
6. **帧结束**：表示遥控帧结束的段。

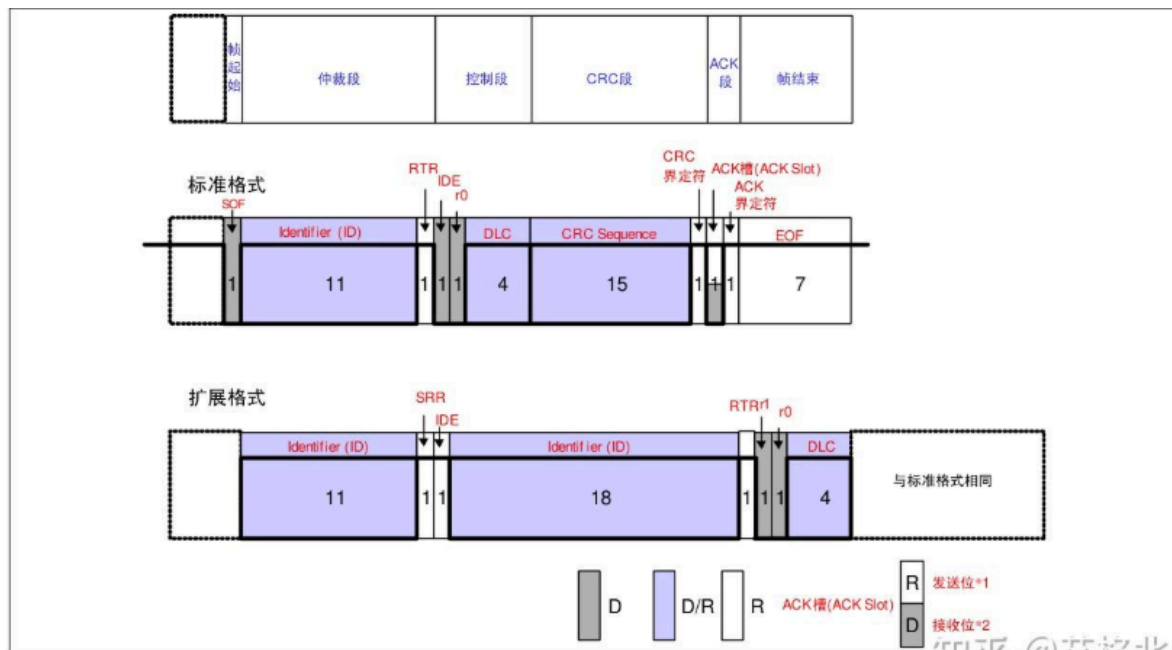


图25. 遥控帧的构成

①、遥控帧和数据帧的不同：

- 遥控帧的RTR位为隐性位(0)，没有数据段；数据帧的RTR位为显性位(1)，有数据段
- 没有数据段的数据帧和遥控帧可以通过RTR位来区分
- RTR位就是用来区分数据帧和遥控帧的

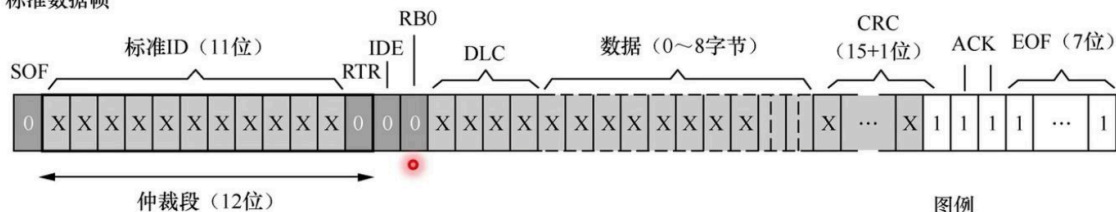
②、遥控帧没有数据段，数据长度码如何表示？

- 遥控帧的数据长度码以所请求的数据帧的数据长度码表示

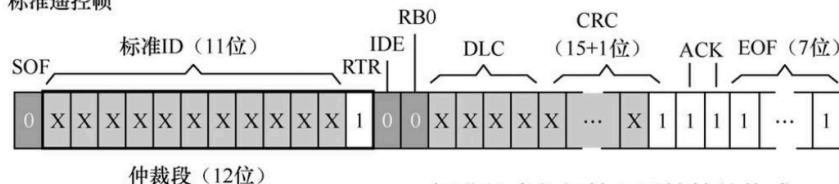
③、没有数据段的数据帧有何用途？

- 可以用于各单元的定期连接确认/应答（心跳、保活）

标准数据帧



标准遥控帧



标准格式数据帧和遥控帧的构成

图例

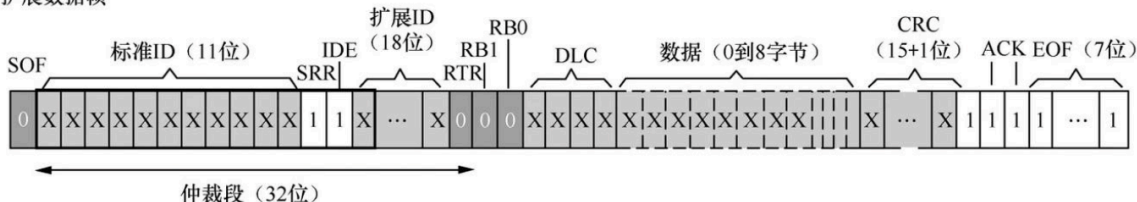
1 隐性电平
(逻辑1)

0 显性电平
(逻辑0)

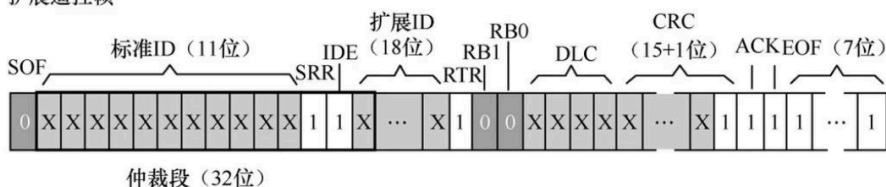
X 显性或隐性

知乎 @艾格北峰

扩展数据帧



扩展遥控帧



扩展格式数据帧和遥控帧的构成

1 隐性电平
(逻辑1)

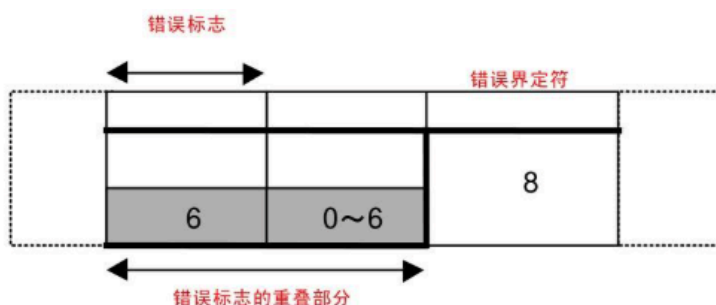
0 显性电平
(逻辑0)

X 显性或隐性

知乎 @艾格北峰

3. 错误帧(Error frame):

用于当检测出错误时向其它单元通知错误的帧



R: 被动错误标志
D: 主动错误标志

R

- 【注】
- *1 主动错误标志
处于主动错误状态的单元检测出错误时输出的错误标志。
 - *2 被动错误标志
处于被动错误状态的单元检测出错误时输出的错误标志。

用于在接收和发送消息时检测出错误通知错误的帧。

错误帧由错误标志和错误界定符构成

(1) 错误标志

错误标志包括主动错误标志和被动错误标志两种。

主动错误标志：6 个位的显性位 (0)

被动错误标志：6 个位的隐性位 (1)

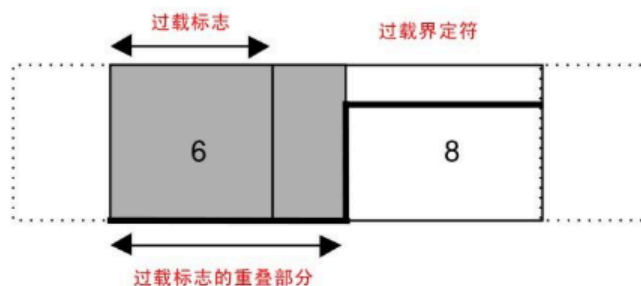
(2) 错误界定符

错误界定符由 8 个位的隐性位构成。

4. 过载帧(Overload frame):

用于接收单元通知其尚未做好接收准备的帧

过载帧由过载标志和过载界定符构成：



(1) 过载标志

6 个位的显性位，过载标志的构成与主动错误标志的构成相同

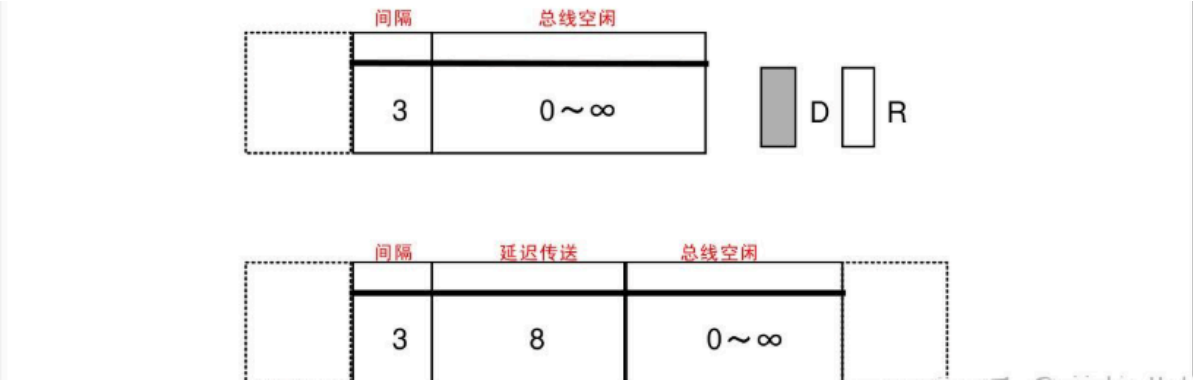
(2) 过载界定符

8 个位的隐性位。过载界定符的构成与错误界定符的构成相同。

5. 帧间隔(Inter-frame space):

帧间隔是用于分隔数据帧和遥控帧的帧。数据帧和遥控帧可通过插入帧间隔将本帧与前面的任何帧（数据帧、遥控帧、错误帧、过载帧）分开

过载帧和错误帧前不能插入帧间隔



(1) 间隔

3 个位的隐性位。

(2) 总线空闲

隐性电平，无长度限制（0 亦可）。本状态下，可视为总线空闲，要发送的单元可开始访问总线。

(3) 延迟传送（发送暂时停止）

8 个位的隐性位。只在处于被动错误状态的单元刚发送一个消息后的帧间隔中包含的段。

八、CAN总线的优先级及同步

1. 优先级的决定

在总线空闲态，最先开始发送消息的单元获得发送权。

多个单元同时开始发送时，各发送单元从仲裁段的第一位开始进行仲裁。连续输出显性电平最多的单元可继续发送。

仲裁的过程如图 所示: 前面的单元1和单元电平都一样，到后面单元1是隐性电平，单元2是显性电平，显性优先级更高，则单元2的优先级更高，获得发送权，而单元1则变为接收状态。

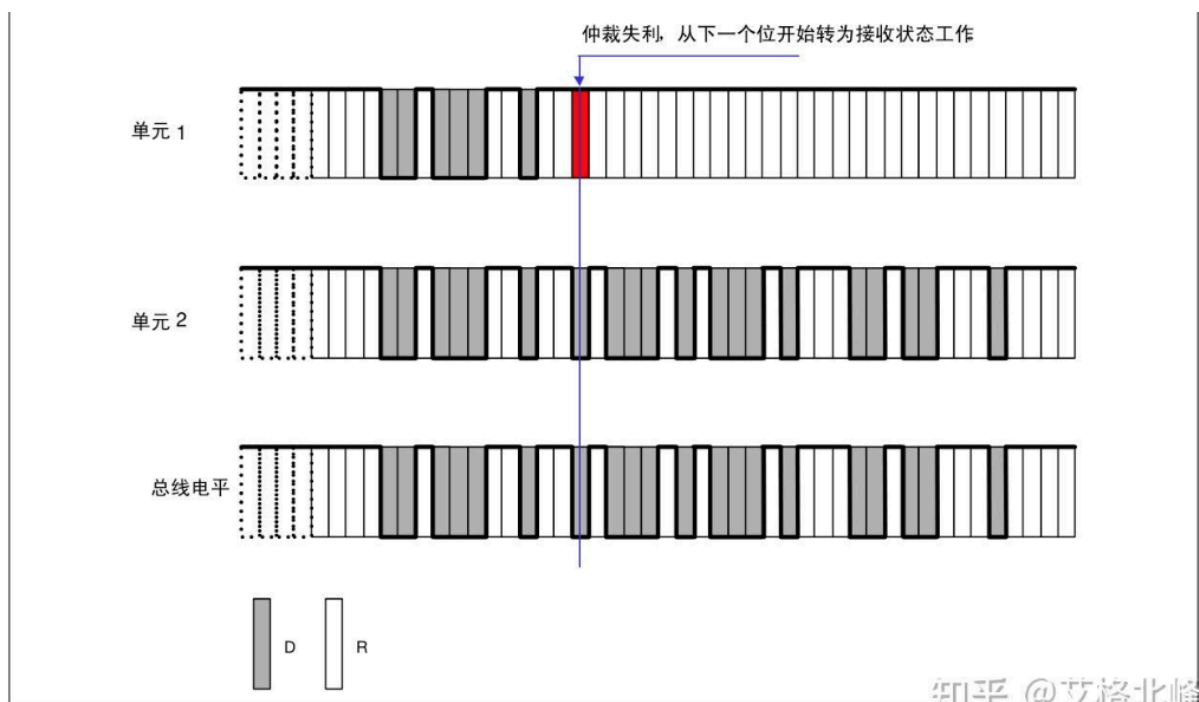


图29. 仲裁过程

2.数据帧和遥控帧的优先级

具有相同 ID 的数据帧和遥控帧在总线上竞争时，仲裁段的最后一位（RTR）为显性位的数据帧具有优先权，可继续发送。

数据帧和遥控帧的仲裁过程如图 所示：

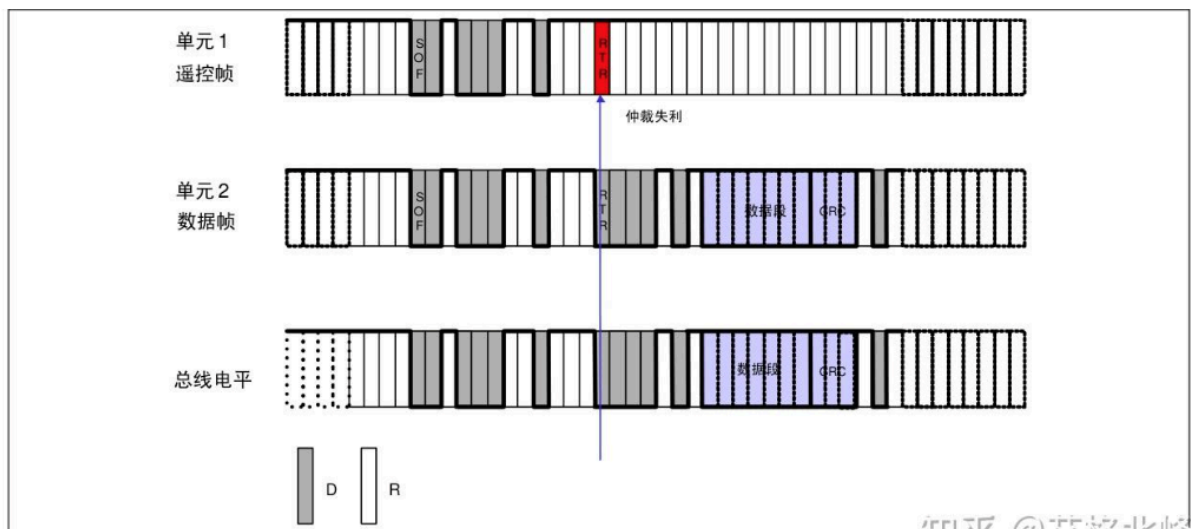


图30. 数据帧和遥控帧的仲裁过程

3.标准格式和扩展格式的优先级

标准格式 ID 与具有相同 ID 的遥控帧或者扩展格式的数据帧在总线上竞争时，标准格式的 RTR 位为显性位的具有优先权，可继续发送。标准格式和扩展格式的仲裁过程如图 所示：

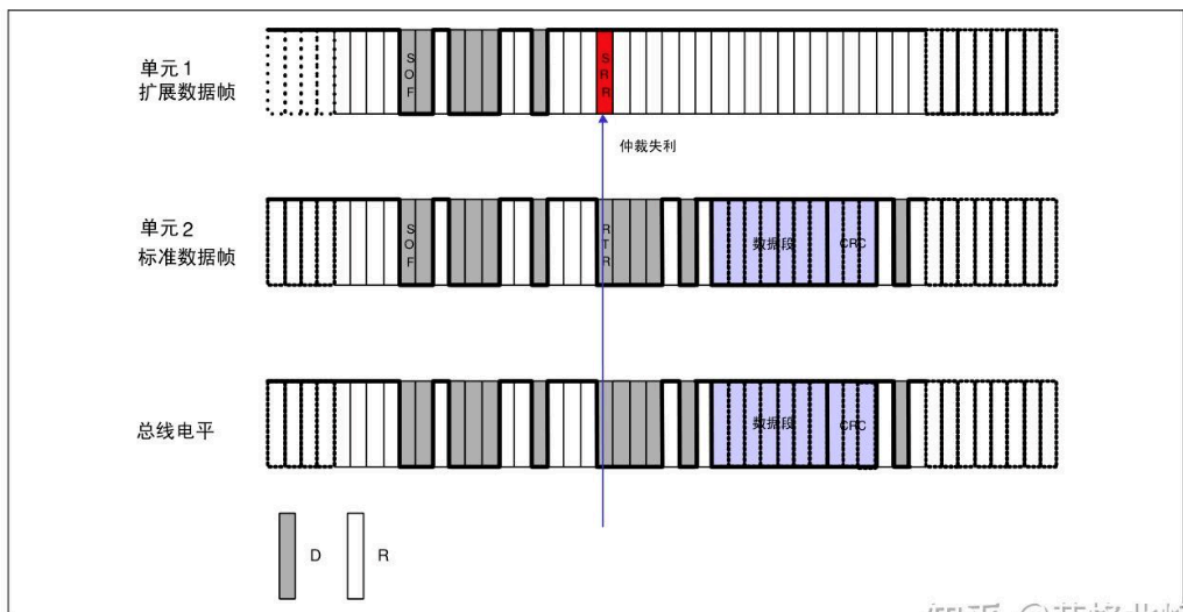


图31. 标准格式与扩展格式的仲裁过程

4. 优先级法则：

数据帧和遥控帧的仲裁段用于多个节点竞争总线时进行仲裁，优先级高的帧获得再总线上发送数据的权利。优先级的确认总结为以下几条法则：

- 在总线空闲时，最先开始发送消息的节点获得发送权；
- 多个节点同时开始发送时，从仲裁段的第一位开始进行仲裁，第一次出现各节点的位电平互异时，输出显性电平的节点获得发送权；
- 相同ID和格式的数据帧和遥控帧，数据帧具有更高优先级，因为数据帧的RTR位时显性电平，而遥控帧的RTR位时隐性电平；
- 对于11位标准ID相同的标志数据帧和扩展数据帧，标准数据帧具有更高的优先级，因为标志数据帧的IDE位时显性电平，而扩展数据帧的IDE位时隐性电平。

5. 错误的种类：

错误共有 5 种。多种错误可能同时发生。

- 位错误
- 填充错误
- CRC 错误
- 格式错误
- ACK 错误 错误的种类、错误的内容、错误检测帧和检测单元如表所示

表6. 错误的种类

错误的种类	错误的内容	错误的检测帧（段）	检测单元
位错误	比较输出电平和总线电平（不含填充位），当两电平不一样时所检测到的错误。	- 数据帧（SOF~EOF） - 遥控帧（SOF~EOF） - 错误帧 - 过载帧	发送单元 接收单元
填充错误	在需要位填充的段内，连续检测到6位相同的电平时所检测到的错误。	- 数据帧（SOF~CRC 顺序） - 遥控帧（SOF~CRC 顺序）	发送单元 接收单元
CRC 错误	从接收到的数据计算出的 CRC 结果与接收到的 CRC 顺序不同时所检测到的错误。	- 数据帧（CRC 顺序） - 遥控帧（CRC 顺序）	接收单元
格式错误	检测出与固定格式的位段相反的格式时所检测到的错误。	- 数据帧 （CRC 界定符、ACK 界定符、EOF） - 遥控帧 （CRC 界定符、ACK 界定符、EOF） - 错误界定符 - 过载界定符	接收单元
ACK 错误	发送单元在 ACK 槽(ACK Slot)中检测出隐性电平时所检测到的错误（ACK 没被传送过来时所检测到的错误）。	- 数据帧（ACK 槽） - 遥控帧（ACK 槽）	发送单元

知乎 @艾格北峰

(1) 位错误

- 位错误由向总线上输出数据帧、遥控帧、错误帧、过载帧的单元和输出 ACK 的单元、输出错误的单元来检测。
- 在仲裁段输出隐性电平，但检测出显性电平时，将被视为仲裁失利，而不是位错误。
- 在仲裁段作为填充位输出隐性电平时，但检测出显性电平时，将不视为位错误，而是填充错误。
- 发送单元在 ACK 段输出隐性电平，但检测到显性电平时，将被判断为其它单元的 ACK 应答，而非位错误。
- 输出被动错误标志（6 个位隐性位）但检测出显性电平时，将遵从错误标志的结束条件，等待检测出连续相同 6 个位的值（显性或隐性），并不视为位错误。

(2) 格式错误

- 即使接收单元检测出 EOF（7 个位的隐性位）的最后一位（第 8 个位）为显性电平，也不视为格式错误。
- 即使接收单元检测出数据长度码（DLC）中 9~15 的值时，也不视为格式错误。

6. 错误帧的输出

检测出满足错误条件的单元输出错误标志通报错误。处于主动错误状态的单元输出的错误标志为主错误标志；处于被动错误状态的单元输出的错误标志为被动错误标志。发送单元发送完错误帧后，将再次发送数据帧或遥控帧。

错误标志输出时序如表所示：

表7. 错误标志输出时序

错误的种类	输出时序
位错误 填充错误 格式错误 ACK 错误	从检测出错误后的下一位开始输出错误标志。
CRC 错误	ACK 界定符后的下一位开始输出错误标志。

7.位时序

由发送单元在非同步的情况下发送的每秒钟的位数称为位速率。

一个位可分为 4 段：

- 同步段 (SS)
- 传播时间段 (PTS)
- 相位缓冲段 1 (PBS1)
- 相位缓冲段 2 (PBS2)

这些段又由可称为 Time Quantum (以下称为 Tq) 的最小时间单位构成。

1 位分为 4 个段，每个段又由若干个 Tq 构成，这称为位时序。

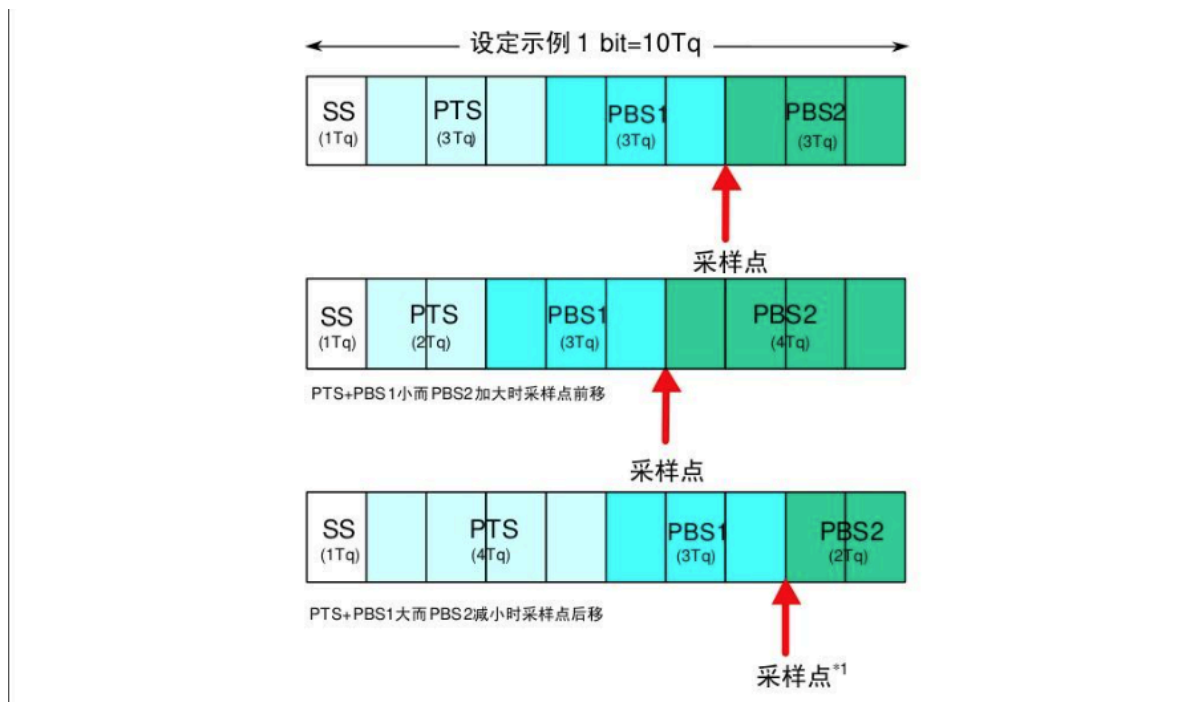
1 位由多少个 Tq 构成、每个段又由多少个 Tq 构成等，可以任意设定位时序。通过设定位时序，多个单元可同时采样，也可任意设定采样点。

各段的作用和 Tq 数如表所示：

表7. 段及其作用

段名称	段的作用	Tq 数	
同步段 (SS: Synchronization Segment)	多个连接在总线上的单元通过此段实现时序调整，同步进行接收和发送的工作。由隐性电平到显性电平的边沿或由显性电平到隐性电平边沿最好出现在此段中。	1Tq	8~25Tq
传播时间段 (PTS: Propagation Time Segment)	用于吸收网络上的物理延迟的段。 所谓的网络的物理延迟指发送单元的输出延迟、总线上信号的传播延迟、接收单元的输入延迟。 这个段的时间为以上各延迟时间的和的两倍。	1~8Tq	
相位缓冲段 1 (PBS1: Phase Buffer Segment 1)	当信号边沿不能被包含于 SS 段中时，可在此段进行补偿。	1~8Tq	
相位缓冲段 2 (PBS2: Phase Buffer Segment 2)	由于各单元以各自独立的时钟工作，细微的时钟误差会累积起来，PBS 段可用于吸收此误差。 通过对相位缓冲段加减 SJW 吸收误差。（请参照图 34）。SJW 加大后允许误差加大，但通信速度下降。	2~8Tq	
再同步补偿宽度 (SJW: reSynchronization Jump Width)	因时钟频率偏差、传送延迟等，各单元有同步误差。SJW 为补偿此误差的最大值。	1~4Tq	

1 个位的构成如图所示：



【注】 *1 采样点

所谓采样点是读取总线电平，并将读到的电平作为位值的点。位置在 PBS1 结束处。

8. 同步的方法：

CAN 协议的通信方法为 NRZ（Non-Return to Zero）方式。各个位的开头或者结尾都没有附加同步信号。发送单元以与位时序同步的方式开始发送数据。另外，接收单元根据总线上电平的变化进行同步并进行接收工作。

但是，发送单元和接收单元存在的时钟频率误差及传输路径上的（电缆、驱动器等）相位延迟会引起同步偏差。因此接收单元通过硬件同步或者再同步的方法调整时序进行接收。

(1) 硬件同步

接收单元在总线空闲状态检测出帧起始时进行的同步调整。在检测出边沿的地方不考虑 SJW 的值而认为是 SS 段。硬件同步的过程如图所示：

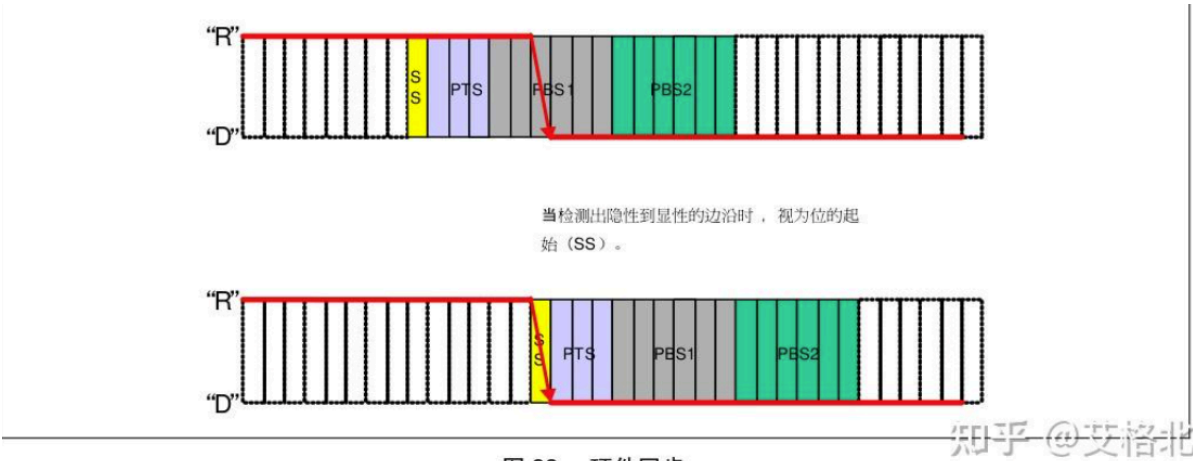
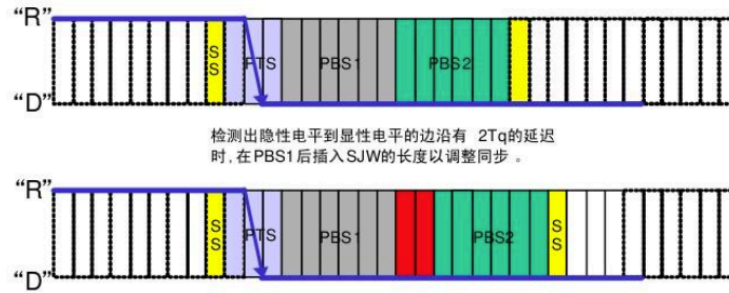


图 33. 硬件同步

(2) 再同步

在接收过程中检测出总线上的电平变化时进行的同步调整。每当检测出边沿时，根据 SJW 值通过加长 PBS1 段，或缩短 PBS2 段，以调整同步。但如果发生了超出 SJW 值的误差时，最大调整量不能超过 SJW 值。再同步如图所示：

隐性电平到显性电平的边沿出现在PTS和PBS1之间时 ($SJW = 2$)



隐性电平到显性电平的边沿出现在PBS2中时 ($SJW = 2$)

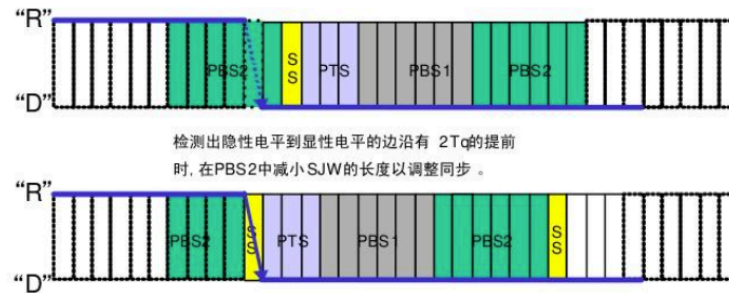


图 34. 再同步

9. 调整同步规则:

硬件同步和再同步遵从如下规则:

1. 1 个位中只进行一次同步调整。
2. 只有当上次采样点的总线值和边沿后的总线值不同时, 该边沿才能用于调整同步。
3. 在总线空闲且存在隐性电平到显性电平的边沿时, 则一定要进行硬件同步。
4. 在总线非空闲时检测到的隐性电平到显性电平的边沿如果满足条件 (1) 和 (2), 将进行再同步。但还要 满足下面条件。
5. 发送单元观测到自身输出的显性电平有延迟时不进行再同步。
6. 发送单元在帧起始到仲裁段有多个单元同时发送的情况下, 对延迟边沿不进行再同步。